

Modules 13–16: ML Deployment › Generative AI › Transformers & LLMs › LangChain

PROGRAM	ASSESSMENT CODE	TOTAL TIME	STACK
Data Science	M16-A1	4 Hours	Python 3

GENERAL INSTRUCTIONS

Attempt all sections. Write your code in Python 3 unless stated otherwise. For coding tasks, submit your .py source file along with a screenshot of your program output. For Section A, write your answers as short paragraphs (4–6 lines each). Label every task clearly with its section and task number. All work must be your own.

A Section A — Concept Application

Answer all questions. Each question presents a real-world scenario followed by a concept-specific question. Modules 13–16 are all in scope: ML Deployment, Generative AI, Transformers & LLMs, and LangChain.

SCENARIO

S
1

You are deploying a delivery time prediction model for a food delivery startup. The data science team trained the model in a Jupyter notebook using scikit-learn, and you need to ship it as part of a production Flask service that will handle several hundred requests per hour.

Question: Compare Pickle and Joblib as serialisation choices for this deployment context. Which would you use and why? What risks does each carry when the model is loaded back in an environment that differs from the one it was trained in?

SCENARIO

S
2

You are working on a food delivery app that wants to auto-generate high-quality promotional images of dishes using only a restaurant's text description and a reference photo of the plated meal. The marketing team wants realistic, visually rich results.

Question: A colleague suggests using a Variational Autoencoder (VAE), while another recommends a Stable Diffusion-based model. Compare the two approaches for this image-generation task — address their key architectural differences, output quality, and which is more appropriate for a production marketing pipeline.

SCENARIO

S
3

You are developing an AI assistant for a food delivery platform. The assistant must handle two distinct jobs: (1) answering open-ended questions such as 'What cuisines are trending near me this weekend?', and (2) classifying incoming customer reviews as Positive, Neutral, or Negative for the analytics dashboard.

Question: GPT-style decoder-only models and BERT-style encoder-only models are both available to your team. Which architecture is better suited to each job, and what fundamental property of each architecture makes it the right choice for that specific task?

SCENARIO

S
4

You are building a customer support bot for a food delivery platform. The bot must handle complaint categories it has never been explicitly trained on – such as 'wrong item delivered', 'packaging damaged', and 'missing cutlery'. Your team has only 3 annotated complaint-response pairs available per category.

Question: Explain the difference between zero-shot and few-shot prompting. Given the constraints described, which strategy would you apply, and how would you structure the prompt to maximise the quality of the bot's responses?

SCENARIO

S
5

You are designing a LangChain-based food ordering assistant. A simpler alternative is a single LLM call that takes the user's message and returns a response directly. Your team is debating whether to build a full agent with tools or use a straightforward LLMChain.

Question: Describe what distinguishes a LangChain agent from a simple LLM chain. Given that your food ordering assistant must check live restaurant availability, call an API to calculate estimated delivery time, and validate discount codes, justify which approach is more appropriate – and identify two risks the agent approach introduces.

SCENARIO

S
6

You are optimising a food delivery chatbot that must accurately answer questions about the platform's 500-page policy document and a menu catalogue that is updated daily. Your team is evaluating two approaches: fine-tuning the underlying LLM on this data, or implementing a Retrieval-Augmented Generation (RAG) pipeline.

Question: Compare fine-tuning and RAG for this specific scenario. Address update frequency, cost, hallucination risk, and data privacy. Which approach is more suitable for a daily-updated, document-grounded Q&A bot, and what limitations does your recommended approach still carry?

B Section B — Practical Coding Tasks

Complete all tasks. Each task targets one or more specific concepts from Modules 13–16. The application domain for this variant is Food Delivery. Follow the requirements listed under each task.

Task 1: Save & Load a Delivery Time Predictor

Train a simple scikit-learn regression model on dummy delivery data, serialise it to disk with Joblib, reload it, and confirm the saved and reloaded model produce identical predictions. (Foundational — single Module 13 concept: model serialisation and reload.)

- Generate a small dummy dataset (100 rows minimum) with features `distance_km`, `num_items`, and `rain_flag`, and a target column `delivery_time_min`; split it 80/20 for training and testing.
- Train a LinearRegression model and print its RMSE on the test split before saving.
- Save the trained model to `delivery_model.joblib` and print a confirmation message that includes the file size in KB.
- Reload the model from disk, run a prediction on one new sample, and assert that the reloaded model's output matches the original model's output for the same input — printing PASS or FAIL accordingly.

Task 2: Flask REST API for Order Predictions

Wrap the saved delivery time model in a Flask REST API so a mobile app or front-end can request live delivery estimates via HTTP. (Applied — Module 13: model loading combined with Flask API design.)

- Load `delivery_model.joblib` once at application startup — not inside the route handler — and keep it in memory for the lifetime of the process.
- Expose a POST endpoint at `/predict` that accepts a JSON body with the fields `distance_km`, `num_items`, and `rain_flag`.
- Return a JSON response containing `predicted_delivery_time_min` rounded to one decimal place and an HTTP 200 status code on success.
- Return a descriptive JSON error body with HTTP 400 if any required field is missing from the request, without crashing the server — demonstrate this by testing with an incomplete payload using a curl command or Postman screenshot.

Task 3: LLM-Powered Dish Description Generator

Build a Streamlit app that sends a structured prompt to an LLM (Ollama local model or any OpenAI-compatible endpoint) and generates an appetising promotional description for any dish on a food delivery menu. (Intermediate — Module 15: prompt engineering and LLM API integration combined with Streamlit UI.)

- Provide two text input fields — Dish Name (e.g. 'Paneer Tikka Masala') and Cuisine Type (e.g. 'North Indian') — and a selectbox for Description Length with options Short, Medium, and Long.
- Construct a structured prompt that includes the dish name, cuisine type, length preference, and an explicit instruction to write in an appetising, customer-facing tone; print the final prompt to the terminal for debugging.
- Display the LLM's response in a styled `st.container()`; show an `st.spinner()` with the message 'Generating description...' while the API call is in progress.
- Display the character count of the generated description below the response box and include a button labelled 'Regenerate' that re-triggers the same API call without the user having to re-enter inputs.

Task 4: LangChain Food Order Assistant with Memory

Build a multi-turn LangChain conversational chain that remembers a user's delivery address and dietary preferences across a session so they do not need to repeat themselves with every message. (Integrative — Module 16: LangChain chains, ConversationBufferMemory, and custom system prompts combined into a working multi-turn flow.)

- Initialise a `ConversationBufferMemory` object and attach it to a `ConversationChain` using an LLM of your choice (Ollama or OpenAI-compatible); verify the `memory_key` matches the variable in the prompt template.
- Write a system prompt that defines the assistant as a helpful agent for 'QuickBite Food Delivery', instructing it to remember the user's address and dietary preference once stated and reference them in later turns.
- Simulate a conversation of at least five turns: the user states their address, states a dietary preference (e.g. vegetarian), asks for a recommendation, asks a follow-up that can only be answered correctly using earlier context, and then asks an unrelated question to test that context is still retained.
- Print each turn in the format 'User: ...' / 'QuickBite: ...' and implement an 'exit' command that ends the loop, prints the total number of turns, and dumps the full memory buffer to the console.

C

Section C — Mini Capstone Project

Mini Project: QuickBite AI — Intelligent Food Delivery Assistant

Objective:

Build a Streamlit-based intelligent assistant for a food delivery platform called QuickBite that combines a deployed ML model (Module 13), prompt engineering and LLM integration (Module 15), and a LangChain agent with memory and a custom tool (Module 16) into one unified, working application.

Your project must:

- The app must include a sidebar where the user enters their delivery address and dietary preference once; these values must persist for the entire session and be automatically injected as context into every LangChain chain call.
- Implement a custom LangChain tool named `get_delivery_estimate` that calls your local Flask /predict API (Task B2) with distance, item count, and weather inputs, and returns a human-readable delivery time string; the agent must invoke this tool automatically when the user asks how long their order will take.
- Add a menu search feature: load a JSON file containing at least 10 menu items (each with name, cuisine, price, and veg flag), filter items based on the user's dietary preference and any cuisine keyword they mention, and inject the top 3 matching results into the prompt context so the LLM can make a personalised recommendation.
- Use a few-shot prompt with at least 2 complete example exchanges in the system prompt to demonstrate the tone, brevity, and format of responses QuickBite should produce.
- Display the full conversation history in the Streamlit UI in a chat-bubble layout (user bubbles right-aligned, assistant bubbles left-aligned) and add a 'Clear Chat' button that resets the LangChain memory and the displayed history simultaneously.

D Section D — AI-Augmented Learning

STEP 1 • BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a Python program that:

- Implements a LangChain ConversationChain for a food delivery order-tracking assistant, where the user can ask for the status of any order by providing an order ID.
- Simulates order status lookup by reading from a local Python dictionary that maps order IDs (e.g. '#101', '#102') to status strings (e.g. 'Preparing', 'Out for delivery', 'Delivered').
- Uses ConversationBufferMemory so that if the user asks 'What about order #102?' immediately after asking about '#101', the assistant responds in context without re-asking the user to clarify which order they mean.
- Runs in a terminal input loop and exits cleanly when the user types 'quit', then prints a summary showing how many unique order IDs were queried during the session.

STEP 2 • TEST & DEBUG (WITHOUT AI)

Then, working without AI, test the code and find at least one bug, limitation, or improvement in the AI's solution. Common issues include: incorrect memory `input_key` or `memory_key` configuration causing a `ValueError`; use of deprecated LangChain chain classes (e.g. `ConversationalChain` instead of `ConversationChain`); missing handling when the order ID is not found in the dictionary; or the chain ignoring memory context on follow-up turns because the prompt template does not include the `{history}` variable. Fix the issue yourself.

SUBMIT

All 3 items required

- 1 The exact prompt(s) you gave the AI tool.
- 2 The AI's original code and your corrected version, with changed lines clearly commented explaining what was wrong.
- 3 A 3–4 line note explaining what you changed and why the AI's version needed it.